Lab 2 TDD i konto bankowe

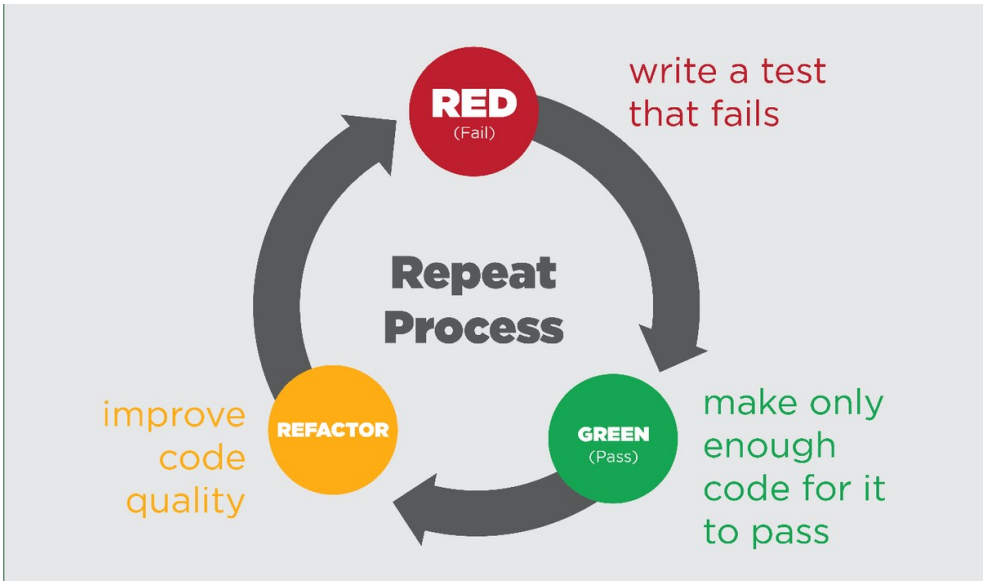
1 Cel zajęć

Zaczynamy projekt bankowy, który rozwijamy przez cały semestr. Metodą TDD napiszesz pierwsze funkcje klasy `Account`, a pracę poprowadzisz w Gitcie aż do pull requesta.

- **Po zajęciach:** przejdziesz cykl Red → Green → Refactor na własnym kodzie.
- Poprosisz AI o review i sam zdecydujesz, które uwagi warto adresować.

Kod, nazwy testów, commity, prompty i opis PR piszemy **po angielsku**.

2 TDD w pigułce



- Najpierw test, który pada. Potem najprostszy kod, który go naprawia. Na końcu porządki przy zielonych testach.
- Jeden test sprawdza jedno zachowanie, a nazwa mówi jakie: `test_too_short_promo_code_is_rejected`.
- Układ testu: Arrange (dane), Act (wywołanie), Assert (sprawdzenie). Zobacz pierwszy test w `tests/unit/test_account.py`.

3 Repozytorium i Git

Aplikacja zapisów: tu tworzysz swoje repozytorium
https://zapisy-1031605285283.europe-central2.run.app

1. Wejdź na link i zaloguj się przez GitHub.
2. Podaj **numer indeksu** i **hasło**, które podam na zajęciach.
3. Aplikacja utworzy prywatne repozytorium `bank-<numer indeksu>` w organizacji `testowanie-2026` i zaprosi Cię do niego. Jeśli GitHub poprosi o zaakceptowanie zaproszenia, zrób to.

```
git clone https://github.com/testowanie-2026/bank-<index number>.git
cd bank-<index number>
python3 -m venv .venv             # once per repository
source .venv/bin/activate         # Windows: .venv\Scripts\activate
python -m pip install -r requirements.txt
python -m pytest                  # 1 failed (the first test is red)
git switch -c lab2                # work on a separate branch
```

Po co venv?
Środowisko wirtualne to prywatny katalog z Pythonem i bibliotekami tylko dla tego projektu. Nowsze systemy (np. Homebrew na macOS) blokują `pip install` do Pythona systemowego (`externally-managed-environment`), więc nie używaj `--break-system-packages`. Aktywację (`(.venv)` w terminalu) powtarzasz w każdym nowym terminalu. Katalog `.venv` jest w `.gitignore`.

Commit po każdej skończonej funkcji, po angielsku, np. `Add PESEL validation to Account`. Na końcu `git push -u origin lab2`.

4 Pliki w repozytorium

Plik	Po co
`src/`, `tests/unit/`	kod i testy. `src/account.py` to pusty szkielet klasy `Account`. `tests/unit/test_account.py` ma pierwszy test, który na starcie pada (to Twój Red).
`.github/workflows/tests.yml`	CI: GitHub Actions uruchamia testy po każdym pushu i w pull requeście.
`AGENTS.md`, `CLAUDE.md`, `GEMINI.md`	zasady dla agentów AI (Claude Code, Gemini CLI): agent **tylko ocenia kod, nie pisze go**. `CLAUDE.md` i `GEMINI.md` wskazują na `AGENTS.md`. Kod i testy piszesz sam, a pliki z zasadami będziemy rozbudowywać na kolejnych zajęciach.

5 Features

Każde zadanie to osobny cykl Red → Green → Refactor i osobny commit.

5.1 Konto i PESEL

Konto zakładamy, podając imię, nazwisko i PESEL: `Account(first_name, last_name, pesel)`. PESEL jest zawsze wymagany. Konto ma też pole `balance`, które na starcie wynosi 0 i **nie jest parametrem konstruktora**.

W repozytorium czeka pusty szkielet klasy i pierwszy test, który pada:

```
class Account:
    def __init__(self):
        pass
```

Uzupełnij konstruktor tak, żeby test przeszedł (Green).

Zastanówmy się nad PESELem. Poprawny PESEL to **dokładnie 11 cyfr**. Inny → pole `pesel` ma wartość `"Invalid"`. Sumy kontrolnej nie sprawdzamy.

Napisz kolejne testy, każdy najpierw czerwony: saldo początkowe 0, PESEL za krótki, za długi i z literą w środku.

Zrobione, gdy: 5 testów zielonych i commit.

5.2 Kod promocyjny

Nasz bank chce zachęcić nowych klientów i zostaliśmy poproszeni o wprowadzenie promocji przy zakładaniu nowego konta.
Konstruktor dostaje opcjonalny `promo_code` (domyślnie `None`). Poprawny ma format `PROM_` + dokładnie 3 znaki (np. `PROM_XYZ`) i **dodaje 50 zł** do salda. Testy: poprawny, brak, za krótki, za długi, zły prefiks.

Zrobione, gdy: 5 testów zielonych, sprawdzanie kodu w osobnej funkcji (Refactor) i commit.

5.3 Promocja tylko dla młodszych

Z kodu korzysta za dużo starszych osób, promocja zaczyna zbyt dużo kosztować nasz bank. Dochodzi nowe wymaganie: 50 zł dostają tylko osoby **urodzone po 1960 r.** (od 1961). Rok urodzenia czytasz z PESEL w funkcji `birth_year(pesel)`. Niepoprawny PESEL → brak promocji. Testy z 5.2 trzeba zaktualizować, bo wymaganie się zmieniło.

1. Napisz testy dla PESEL-i z roku 1990, 1959 i 1961 i doprowadź je do zielonego.
2. Dopisz test dla osoby z 2001 r. Co się dzieje i dlaczego?
3. Napraw kod.

Fikcyjne PESEL-e do testów (liczy się format): `90010112345` (1990), `59010112345` (1959), `61010112345` (1961), `01210112345` (2001).

> **Zrobione, gdy:** testy dla 1990, 1959, 1961, 2001 oraz dla niepoprawnego PESEL-u zielone, `birth_year` ma własne testy i jest commit.

6 Review od AI i pull request

AI jest recenzentem, nie autorem. Jego uwagi to materiał do namysłu, a decyzja należy do Ciebie.

1. **Review.** W folderze projektu uruchom `claude` lub `gemini` (zasady z `AGENTS.md` wczytają się same) i wklej prompt. Zwykły czat? Wklej wynik `git diff main` pod promptem.

```
Review the changes on the lab2 branch compared to main.
Review both the tests and the code.
Do not write or fix any code, only point out problems. If code looks ok say so.
```

2. **Decyzja.** Przy każdej uwadze zdecyduj, czy ją zaadresujesz. Sprawdź, czy jest prawdziwa (AI myli się przekonująco), czy mieści się w zakresie zajęć i ile kosztuje poprawka.
3. **Poprawka.** Wybierz jedną uwagę i zaadresuj ją w TDD: najpierw test, potem kod, osobny commit.
4. **Opis PR.** Poproś AI o opis, sprawdź go i popraw

```
Write a pull request description in English for the changes on the lab2
branch compared to main. Include: a short summary, a list of changes,
and how they were tested. Describe what changed. Do not review or
grade the code.
```

5. **Pull request.** Stwórz PR i sprawdź czy wszystko jest OK. Jeżeli jest sprawdź pipeline i zrób merge

> **Zrobione, gdy:**
> - PR `lab2 → main` jest otwarty, a jego opis sprawdziłeś i poprawiłeś,
> - CI (Actions) jest zielone.
> - Merge zrobiony

7 Gdy coś nie działa

- `No module named 'src'` lub `no tests ran`: uruchamiaj `python -m pytest` (z aktywnym venvem) z głównego folderu projektu. Testy leżą w `tests/unit/test_*.py`, a `src/` ma `__init__.py`.
- Actions się nie uruchamiają: sprawdź zakładkę Actions i czy `.github/workflows/tests.yml` jest na wypychanej gałęzi.
- Brak `claude` lub `gemini`: użyj zwykłego czatu i wklej `git diff main` z promptem.

Do poczytania: Test Driven Development (Martin Fowler), pytest, AGENTS.md.